LANGGRAPH'S RUNTIME

Five views of what LangGraph does under a graph: the superstep, the Send fan-out, the reducer merge, the checkpoint tree and the interrupt that re-runs a node. Each is drawn from a JSON record of a real run.

1.1 WHAT THIS IS.

`:::loop` draws an agent loop as its author sees it: nodes, edges, a run. These five blocks draw the same kind of graph as the runtime sees it: tasks, channels, versions, checkpoints. They take JSON, so a coding agent can record a run and get the figure.

Table 1-1. The blocks

Block	Reads	Shows
`:::superstep`	a pregel trace	one superstep at a time: tasks, writes, channel versions, versions_seen, why each node runs next
`:::fanout`	the same trace	tasks by superstep, Sends into PUSH tasks, the gather, a deferred node waiting
`:::reducer`	the same trace, `step=` and `channel=`	one step's writes to one channel, folded in merge order
`:::checkpoints`	a thread's history	checkpoints as a tree: parents, forks, pending writes, interrupts, the head
`:::interrupt`	a run script	interrupt() and Command(resume): the node re-running from its first line

One trace feeds three views, so an example is recorded once.

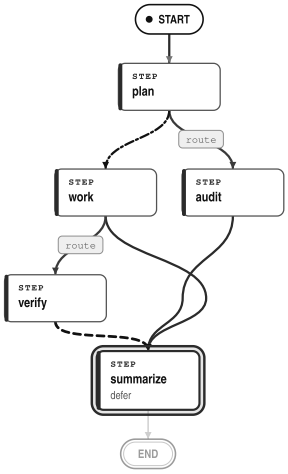
1.2 A SUPERSTEP AT A TIME.

The graph plans three subjects, sends each to `work` with `Send`, and audits the plan in the same superstep. One subject needs a second look in `verify`. A deferred `summarize` gathers everything. Step through it: the graph lights the tasks that run, the channel table shows which versions moved, and the last panel says why each node does or does not run next.

PREGEL SUPERSTEP

Map-reduce with a deferred summary

PULL task · PUSH task (a Send) · channel updated · runs next



TASKS THAT RAN, IN MERGE ORDER

1 **PULL** **summarize**

triggered by `branch:to:summarize`

```
summary "5 results"
```

CHANNELS AFTER THE STEP

CHANNEL	VERSION	VALUE
`results` BinaryOperatorAggregate (add)	5	`["audit: 3 subjects","moon: done","sun:…`
`subjects` LastValue	3	`["moon","sun","sea"]`
`summary` LastValue	**6**	`"5 results"` by summarize
`topic` LastValue	2	`"tides"`
~~`branch:to:audit`~~ EphemeralValue	4	empty
~~`branch:to:plan`~~ EphemeralValue	3	empty
~~`branch:to:summarize`~~ LastValueAfterFinish	**5 → 6**	empty
~~`branch:to:verify`~~ EphemeralValue	5	empty
`__pregel_tasks` Topic	4	set
~~`__start__`~~ EphemeralValue	2	empty

VERSIONS_SEEN, AND WHAT RUNS NEXT

next: **nothing; the run ends**

NODE	TRIGGER: VERSION / SEEN	
plan	branch:to:plan *3 / 2*	newer, but empty
audit	branch:to:audit *4 / 3*	newer, but empty
work	no trigger channel: runs only from a Send	
verify	branch:to:verify *5 / 4*	newer, but empty
summarize	branch:to:summarize *6 / 5*	newer, but empty

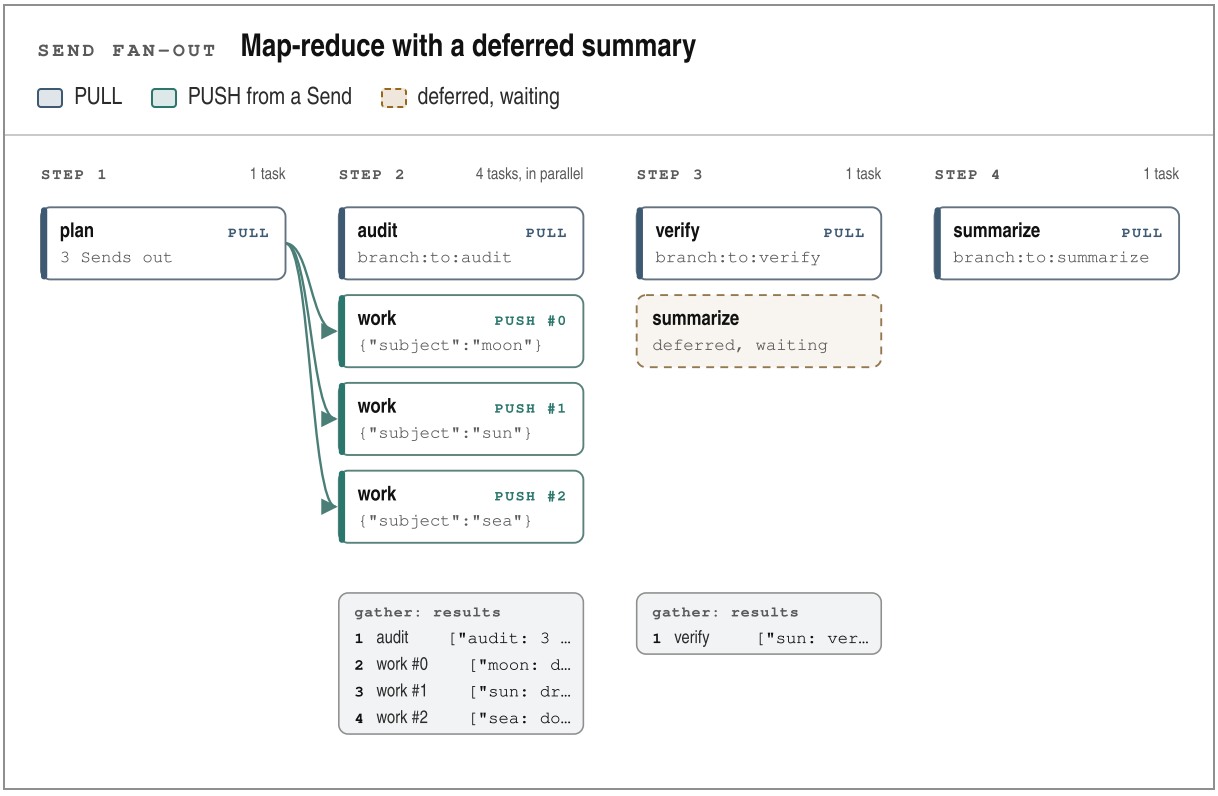
Figure 1-1. Map-reduce with a deferred summary, one superstep at a time

PREGEL-501A

PREGEL

Three things to watch. In step 1, `plan` writes three Sends to `__pregel_tasks`: they become the PUSH tasks of step 2, and `work` never appears in `versions_seen`, because a PUSH task has no trigger channel. After step 1, `branch:to:plan` has a newer version than `plan` saw, but it is empty: clearing an ephemeral channel bumps its version, and only a channel holding a value can trigger. After step 2, `branch:to:summarize` holds a value, but `summarize` is deferred: it waits until no other task is due, so `verify` runs alone in step 3.

1.3 THE FAN-OUT.



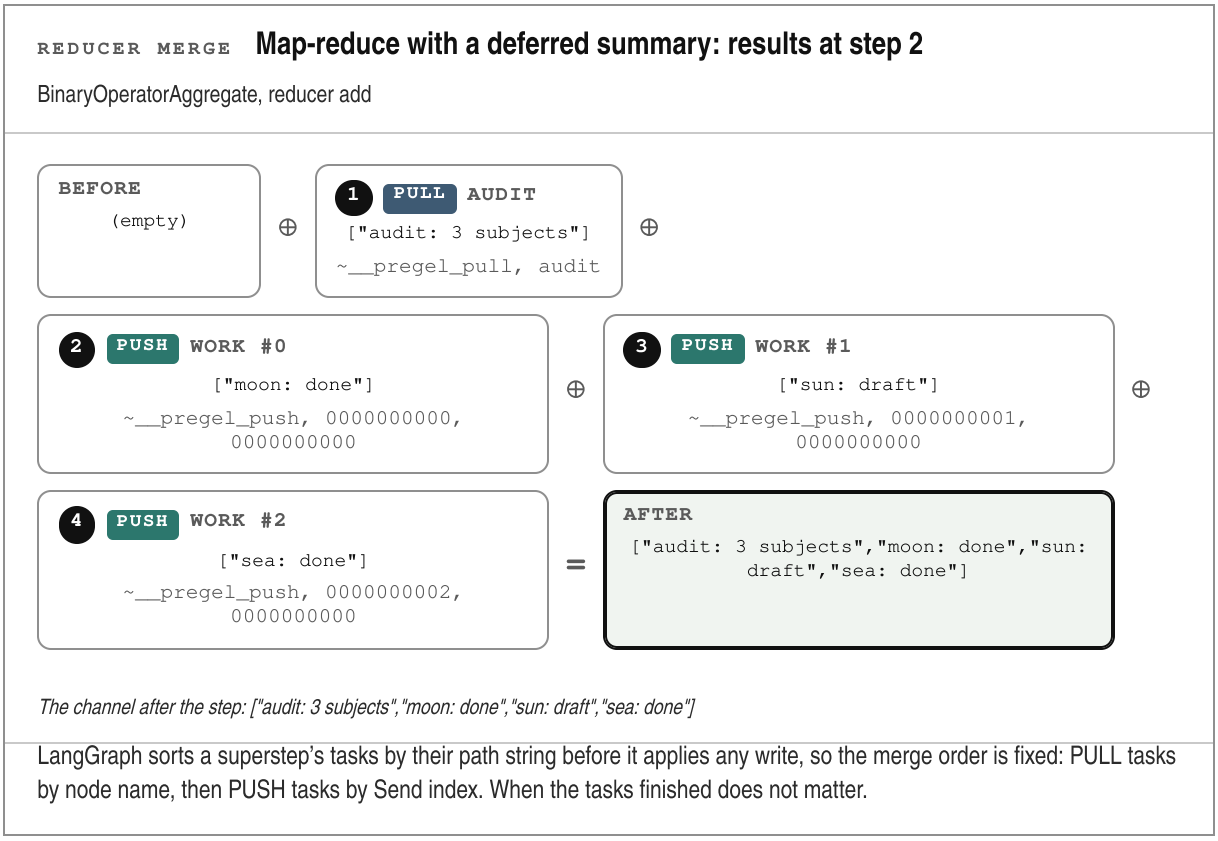
PREGEL-502A

Figure 1-2. Map-reduce: three Sends, a gather in path order, a deferred node

Hover a task to see its Sends and where its writes land in the gather.

1.4 THE MERGE.

Four tasks write to `results` in step 2. LangGraph sorts a step's tasks by their path before it applies a single write: `~__pregel_pull, audit` comes before `~__pregel_push, 0000000000, ...`. So the list reads audit, moon, sun, sea, however the tasks finished.



PREGEL-503A

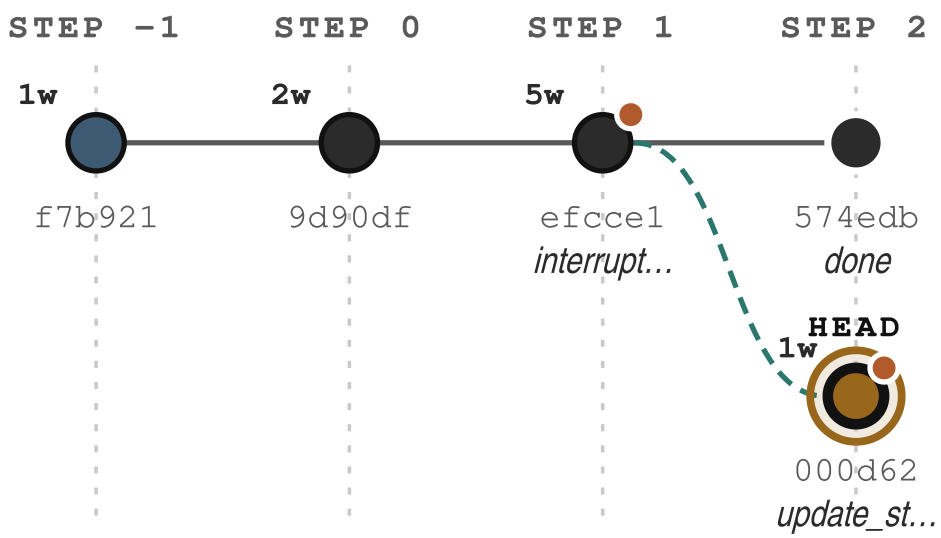
Figure 1-3. Step 2's writes to results, folded in merge order

1.5 THE CHECKPOINT TREE.

A two-node graph: `write` drafts, `review` asks twice with `interrupt()`. It is run, resumed twice, and finishes. Then `update_state` goes back to the checkpoint before `review`, changes the draft, and the graph runs on from there: a fork. The head is the fork's tip.

CHECKPOINT HISTORY **Two interrupts, then a fork thread t**

input loop update fork interrupt HEAD



CHECKPOINT

1f1ddeda-dd20-6482-8002-72b64a000d62

STEP	2
SOURCE	update
PARENT	efcce1
NEXT	review
MADE BY	update_state(draft="v2"), then invoke(None)
VALUES	{ "draft": "v2" }
INTERRUPT	review: {"approve":"v2"}

PENDING WRITES (SAVED AGAINST THIS CHECKPOINT)

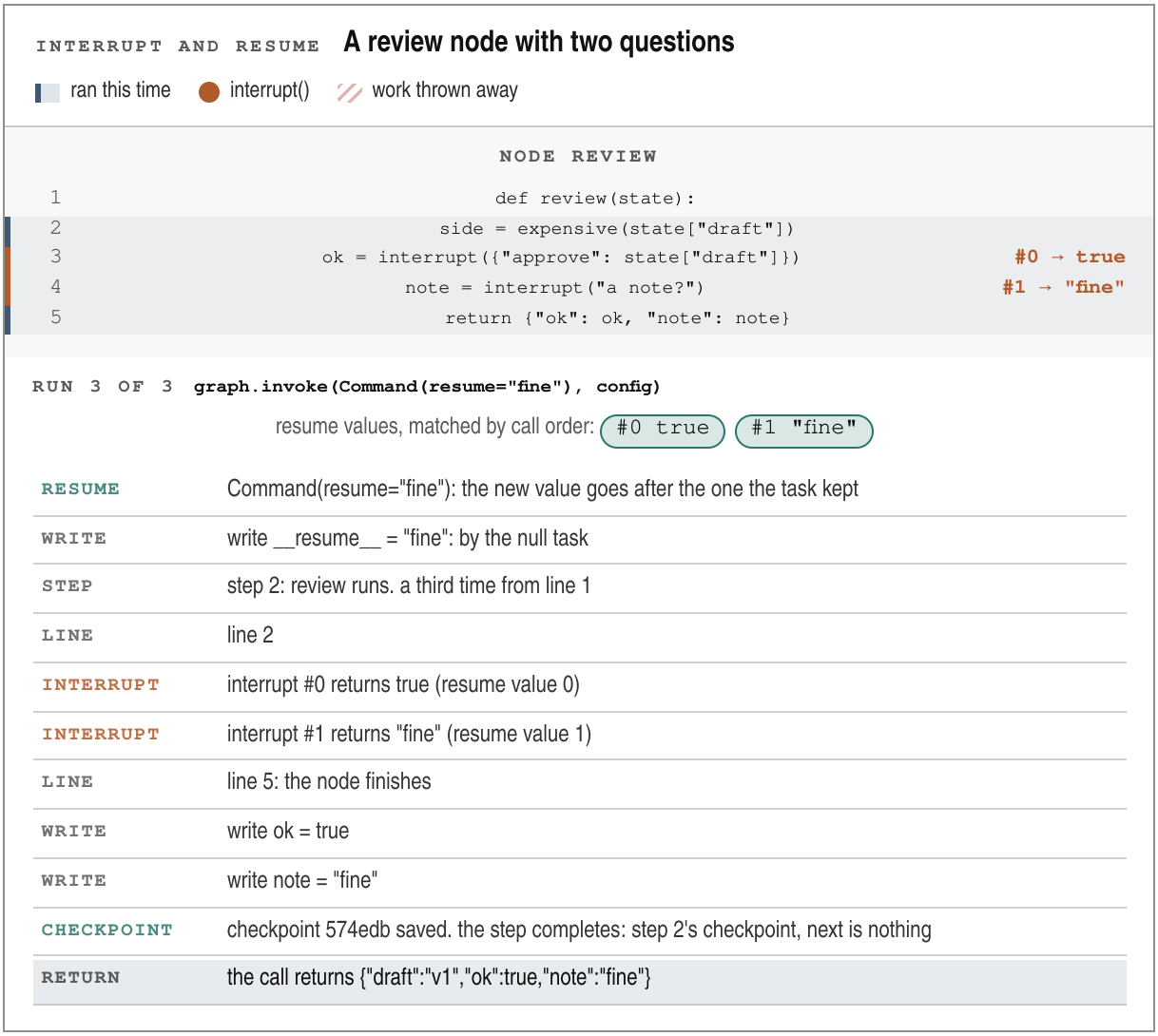
407f12	__interrupt__	[{"value":{"approve":"v2"},"id":"2e46be…

PREGEL-504A

Figure 1-4. Two interrupts, a finished run, and a fork from an earlier checkpoint

Pick the step-1 checkpoint. Its pending writes are the whole story of the interrupts: the null task's `__resume__`, the task's own `__resume__` list, one `__interrupt__` (each new interrupt replaces it), and at the end the node's real writes. All of them are saved against the checkpoint the task started from, not a new one.

1.6 INTERRUPT AND RESUME.



PREGEL-505A

Figure 1-5. interrupt() twice, Command(resume) twice: the node starts over each time

What the real run showed. The node runs from line 1 on every resume, so anything before an interrupt runs again: keep it idempotent, or move it to its own node. Resume values are matched by the order of `interrupt()` calls, not by name. Since 1.2.20 each `interrupt()` call in a node has its own id; through 1.2.19 they shared one.

1.7 THE FORMATS.

`pregel/capture.py` writes the first two from a real run: `trace(graph, input, config)` and `history(graph, config)`, for a graph compiled with an `InMemorySaver`. The interrupt script is written by hand, because only the author knows which line is which.

Table 1-2. A pregel trace (superstep, fanout, reducer)

Field	What
`title`, `channels`, `defer`	the channels' types and reducers; the deferred nodes
`graph`	optional: a loopgraph JSON of the graph, which gives the stepper its graph pane
`steps[]`	one per checkpoint: `step`, `source`, `checkpoint`, `input` (the first only)
`steps[].tasks[]`	the tasks that produced this checkpoint: `name`, `kind` pull or push, `triggers`, `path`, `writes` as `[channel, value]`, `status`, optional `ms`
`steps[]` after the tasks	`updated`, `channel_versions`, `versions_seen`, `available`, `values`, `next`, optional `note`

Table 1-3. A history (checkpoints)

Field	What
`thread`, `head`	the thread, and the checkpoint `get_state` returns
`checkpoints[]`	`id`, `parent`, `step`, `source`, `next`, `values`, `pending_writes` as `[task_id, channel, value]`, `tasks` with `interrupts`, optional `label` and `run`
`runs[]`	optional: `id`, `label`, for checkpoints that name their run

Table 1-4. A run script (interrupt)

Field	What
`node`, `code`	the node's name and its source, one string per line
`runs[]`	`label`, optional `resume` (the values the run can match, in order), `events`
`runs[].events[]`	`kind`: step, line, interrupt, resume, write, discard, checkpoint, return or note; with `line`, `index`, `value`, `resumed`, `channel`, `id`, `pending`, `note` as they apply